A learned, order-preserving open general classification: objective, algorithm, stable identifiers and a first benchmark

David Awad

DRAFT. This is a working draft for internal review. It has not been submitted, posted or shared anywhere. Every number in it is generated by `paper/paper2/scripts/make_tables.py` from the committed results JSON; edit `paper.src.md`, not `paper.md`. Citation keys refer to `references.bib`, the cited subset of `docs/research/references.bib`.

Abstract

Library classifications such as LCC and DDC are maintained by editorial staff, are partly closed, and were not built against a stated, measurable objective. We propose the opposite: a general classification that is *fitted*, under the constraints that make a classification usable on a shelf, to the same quantity that a benchmark of existing schemes ranks them by. The objective is J = D + 0.005 N, where D is the normalized Dasgupta cost of the class tree on a fixed similarity graph (Dasgupta 2016) and N is an expected navigation cost in bits with a per-level charge. Two constraints are hard: every class is a contiguous run of one fixed shelf order (the optimal-alphabetic-tree constraint of Hu and Tucker (Hu and Tucker 1971)), and new works are filed by splitting leaf classes only, so that every existing class keeps its code, label and parent. We describe a deterministic fitting algorithm (average linkage, rotation local search, an exact dynamic-programming collapse to 2-12 children and at most 4 levels, and order-preserving local search), state the identifier-stability guarantee precisely, and tie each clause of it to a test. On a corpus of 10,000 arXiv papers the fitted prototype has 110 hand-labelled classes and reaches cost 0.1722 in sample (average linkage 0.1518; arXiv's own categories 0.3333; the best existing scheme reached through the same lexical classifier 0.6414). Fitted on 80% and tested on the other 20%, it scores 0.1722. Filing 500 library books as new works created 16 classes and changed no existing identifier. Rescored on graphs from two independent embedding models, the scheme still leads every existing scheme in all 9 graph conditions, with a smaller margin. Under a corrected objective that also charges for joining dissimilar works, it still leads on the papers, but on 3,000 library books its lead over librarian-assigned Dewey and LCC numbers depends on the graph setting. These numbers flatter the method: the held-out and holdout filings use the same embedding as the fit, the scheme has seen only arXiv, and labels for inserted classes are weak. We state those limits plainly, release the scheme as CC0 open data, and ask whether a learned, open, order-preserving classification can become a general one.

1. Introduction

1.1 Why a learned general classification

A general classification gives every work one address in one linear order. That order serves two readers: the person browsing a shelf, who wants related works next to each other, and the cataloguer or system that must file each new work somewhere. The schemes libraries use for this, LCC and DDC above all, were enumerated by hand and have been extended editorially since DDC's first edition of 1876 (Comaromi 1976; Hjørland 2025) and LCC's early twentieth-century schedules (Fraser 2020). LCC's schedules are freely downloadable as PDF (Library of Congress 2026b) and LC describes its linked-data service as a public-domain data set (Library of Congress 2026a); the copyright in DDC is owned by OCLC (OCLC, n.d.). Open universal alternatives exist (the Basic Classification (BARTOC, n.d.), the Free Decimal Correspondence (Ockerbloom, n.d.), the Integrative Levels Classification (ISKO Italia, n.d.)), alongside open thesauri with a hierarchy such as the UNESCO Thesaurus (UNESCO, n.d.) and EuroVoc (Publications Office of the European Union, n.d.), but none is both deep and open with a full crosswalk (research report 1 on this branch). None of them, open or closed, was designed against a stated, measurable

objective, so there is no way to say whether one is better than another for a given collection except by argument.

Paper 1 (`bench/results/2026-10-10`, `bench/results/2026-10-10-r2`) measured 13 published schemes on one objective scale: the Dasgupta cost of each scheme's class tree over a fixed kNN similarity graph of 10,000 arXiv papers, with navigation statistics alongside. That benchmark makes the converse question natural. If a scheme can be scored, can a scheme be *fitted* to the score, under the constraints a library needs, and how far does it get?

Data-driven classifications already exist at scale. The CWTS publication-level classification clusters a citation network by a modularity-type function at several resolutions (Waltman and Eck 2012; Traag, Waltman, and Eck 2019), and OpenAlex Topics assigns works to citation-derived topics whose upper levels are mapped from an existing journal classification (Priem, Piwowar, and Orr 2022; OpenAlex 2026). Topic-taxonomy builders such as TaxoGen (Zhang et al. 2018) and LLM-driven pipelines such as Chain-of-Layer and TnT-LLM (Zeng et al. 2024; Wan et al. 2024) produce trees from text. None of these optimizes a shelf order, and none gives identifiers a guarantee under growth. Those two properties are what separates a classification from a clustering for a library.

1.2 Why order preservation matters

A shelf, a call-number index, and a browse list are each one linear order. A hierarchy is useful for them only if every class is a contiguous run of that order, so that "go to class X" means "go to one place". A tree fitted without this constraint can always be drawn in *some* leaf order, but a search that freely regroups works across the order produces classes whose works are not adjacent once the order is fixed. We therefore fix the shelf order first and restrict every later step to trees whose leaf order is that shelf order. This is the alphabetic-tree constraint of optimal order-preserving codes and search trees (Hu and Tucker 1971; Knuth 1971), as opposed to the free leaf order of Huffman coding (Huffman 1952).

1.3 Why identifier stability matters

Class codes are written into call numbers, spine labels, catalogue records, crosswalks and links. A classification that is refitted from scratch each year would renumber works that never moved, invalidating all of those. For a learned scheme this is the decisive practical objection: an optimizer will happily trade any existing code for a small gain in the objective. We therefore separate *fitting* (done once, in batch) from *growth* (done incrementally), and constrain growth so that existing identifiers are never changed, only refined. TaxoExpan (Shen et al. 2020) inserts new nodes into a frozen taxonomy, but we found no objective in the literature that trades optimality against re-shelving (research report 2).

1.4 Contributions

1. An objective for a general classification that combines similarity coherence and navigation cost, with explicit shape constraints, computed by the same harness that scores published schemes (Section 2).
2. A deterministic, order-preserving fitting algorithm, with an exact dynamic program for the collapse of a binary tree to a bounded-shape class tree (Section 3).
3. An insertion procedure for new works with a precisely stated identifier-stability guarantee, checked on every run and by tests on real data (Section 4).
4. A hand-labelled prototype, `scheme:learned@2026-10-10`, released under CC0, and a benchmark against 13 published schemes in sample, through a lexical classifier, on a held-out split, and on a holdout of library books (Sections 5-6, 9).

2. Objective

Let G be a weighted similarity graph over n works, with edge weights w_ij and total weight W. Here G is the published Paper 1 graph: the sentence-transformers/all-MiniLM-L6-v2 embedding (Reimers and Gurevych 2019; Wang et al. 2020; sentence-transformers, n.d.) of title and abstract, cosine similarity minus the mean pair cosine, k = 20 nearest neighbours, symmetrized by maximum (148,111 edges over n = 10,000 works). A class tree T places each work in one leaf class; a virtual root holds the top-level classes.

Similarity term. Works hang under their leaf class, so two works in one class meet at that class and pay its size:

```
D(T) = (1 / (n W)) * sum over edges (i, j) of w_ij * |items(lca_T(i, j))|
```

This is Dasgupta's cost (Dasgupta 2016), normalized so that a flat tree scores 1, and it is exactly the harness's `cost_norm`. Its complement is the Moseley-Wang revenue (Moseley and Wang 2023). Exact minimization is hard; the approximation and axiomatic literature (Charikar and Chatziafratis 2017; Cohen-Addad et al. 2019) and continuous relaxations (Chami et al. 2020) concern binary trees without shape or order constraints, which is why we start from average linkage and add the constraints in later steps. The harness also reports `skill = (1 - D) / (1 - D_ref)`, where D_ref is the cost of the average-linkage tree on the same graph.

Navigation term. The fan-out of a class is its number of child classes, or its number of works if it is a leaf class. Then

```
N(T) = (1 / n) * sum over nodes v with fan-out > 1 of |items(v)| * (a + log2 fanout(v))
```

is the mean, over works, of the cost of descending from the root to the work, charging `a` per menu level plus `log2` of the menu size, a Hick-Hyman-style choice cost (Hick 1952; Hyman 1953). It is the harness's `nav_log_a1`. A balanced tree pays about log2 n bits plus `a` per level, so `a` prices a level and the `log2` part penalizes fan-outs that do not match the sizes below them. Without `a`, depth would be free; without the log term, the cheapest tree would be flat.

Objective and constraints.

```
J(T) = D(T) + lambda * N(T),   lambda = 0.005,  a = 1
```

subject to: every class has 2-12 child classes, at most 4 class levels, leaf classes of at least 5 works, and every class a contiguous run of one fixed shelf order with siblings ordered along it. With these weights, one extra level for every work is worth a 0.005 drop in D: 0.5% of the corpus scanned per related pair.

The two terms pull against each other. D falls with every split and is smallest for a binary tree with one work per leaf; N charges `a` bits per level to every work under it. The choice of lambda, `a` and the depth bound is a modelling decision, not a fitted quantity; Section 6.5 reports a sweep. We do not claim that J is *the* objective for a general classification: the optimal tree is defined relative to the similarity w, and there are formal reasons to doubt any similarity-free optimum (Kleinberg 2002; Carlsson and Mémoli 2010). The claim is narrower: once a benchmark ranks schemes by D and N, a scheme can be fitted to them, and the result is a reference point for every hand-built scheme on the same scale.

3. Algorithm

`learned-scheme build` (`schemes/learned/src/fit.rs`) is deterministic for a fixed seed (20261010, used only for local-search visiting orders).

1. Average linkage on the sparse graph (`src/binary.rs`). Cluster similarity is the summed edge weight over the product of the cluster sizes, missing pairs counting 0; this is Moseley and Wang's average linkage, which approximates the optimal Dasgupta revenue within a factor of 1/3 (Moseley and Wang 2023). Ties go to the lowest cluster ids; disconnected components are joined smallest first. The resulting binary tree's cost, 0.151834, equals the harness's reference value.
2. Rotation local search on D. At a node x = (A, C) with A = (A1, A2), try ((A1, C), A2) and ((A2, C), A1) with exact local deltas. On this graph it makes 0 moves. That is expected: a rotation helps exactly when the average weight between A1 and C exceeds that between A1 and A2, and average linkage is reducible, so when A1 and A2 merged no part of a later C was closer. The pass certifies rotation-optimality and matters for other starting trees.
3. Shelf order. Each merge's two children are oriented so that the child more strongly tied to the works already shelved to its left comes first. The binary tree's leaf order becomes the fixed shelf order; nothing later changes it.
4. Exact collapse (`src/collapse.rs`). A dynamic program over the binary tree chooses which binary nodes become classes, contracting the rest, to minimize J over all such class trees within the shape bounds. For a class u it chooses the frontier of descendant classes by a knapsack over the binary nodes in between. Binary

subtrees with fewer than 5 works are glued to their sibling and join the adjacent leaf class on the shelf. The DP is exact when nothing is glued above leaf level and close otherwise (J 0.264697 by the DP, 0.264730 evaluated exactly here). Every tree this step can produce is order-preserving by construction, because contracting binary nodes never reorders leaves.

5. Order-preserving local search (`src/search.rs`) on exact J: shift a boundary work into the neighbouring leaf class on the shelf (incremental delta), hand the last child class to the next sibling or the reverse, merge adjacent sibling leaves, flatten a class into its parent, group two or three adjacent siblings under a new class. Every move keeps the shape bounds and the shelf order. Rounds repeat until no move improves J.

Table 1 gives the stages on the corpus (from `build-report.json`).

stage	D (cost_norm)	N (nav, a = 1)	J	classes	seconds
average linkage (binary, one work per leaf)	0.151834	-	-	-	0.91
rotation local search (0 moves)	0.151834	-	-	-	0.04
exact collapse (DP)	0.172309	18.4841	0.264730	129	0.05
order-preserving local search (88 shifts, 20 structure moves)	0.172156	18.4487	0.264399	110	1.19

Table 1. Fitting stages on the 10,000-paper corpus. The four stages take 2.2 s of wall time in total on one core.

The result has 110 classes (2 / 5 / 16 / 87 by level), 88 of them leaf classes of 38-345 works (median 98). Class codes are the parent's code plus a two-digit position (`01`, `01.02`, `01.02.03`), so codes sort in shelf order. The root has 2 children; internal classes have at most 8 (mean 4.8). Table 2 shows the top three levels.

code	label (hand-written)	leaf classes	works
`01`	Computing, AI and engineering	50	5,305
`01.01`	Language models, security and computing systems	18	2,555
`01.01.01`	Language, speech and LLM agents	5	1,221
`01.01.02`	Security, privacy and software	6	581
`01.01.03`	Efficient computing, networks and energy systems	7	753
`01.02`	Vision, robotics, dynamics and medicine	22	1,927

code	label (hand-written)	leaf classes	works
01.02.01	Computer vision and multimodal models	4	585
01.02.02	Robotics and control	5	424
01.02.03	Machine learning for physical and dynamical systems	8	371
01.02.04	Medical imaging and health	5	547
01.03	Statistics, economics and decision-making	10	823
01.03.01	Statistical and causal inference	5	290
01.03.02	Financial forecasting and markets	1	216
01.03.03	Decision-making, learning and games	4	317
02	Mathematics and physical sciences	38	4,695
02.01	Mathematics	14	2,251
02.01.01	Applied mathematics, analysis and probability	7	1,105
02.01.02	Pure mathematics	7	1,146
02.02	Physics	24	2,444
02.02.01	Quantum information and quantum computing	6	465
02.02.02	Condensed matter, materials and chemistry	7	836
02.02.03	Astrophysics and particle physics	6	808
02.02.04	Gravitation and theoretical physics	5	335

Table 2. The top three levels of `scheme: learned@2026-10-10`, with the number of leaf classes and fitted works below each class.

The test `rebuild_reproduces_the_committed_scheme_and_matches_the_harness` (`schemes/learned/tests/committed.r` rebuilds the scheme byte for byte from committed inputs and checks that D and N as computed by the builder equal the harness's `cost_norm` and `nav_log_a1` for the committed scheme. `make_tables.py` repeats the second check on the published numbers.

4. Identifier stability under insertion

4.1 Procedure

`learned-scheme insert` (`src/insert.rs`) files new works one at a time without renaming anything.

1. A new work goes to the leaf class whose centroid is nearest (cosine on unit sentence-transformers/all-MiniLM-L6-v2 vectors; the committed centroids are 384-dimensional, i8-quantized, one per leaf class with its work count).

2. A leaf class tries a two-means split, anchored on its fitted centroid, when either (a) it has taken at least 12 new works and they form a distinct group (the two centroids' cosine is at most 0.5), or (b) its new works reach its capacity: its fitted size, or the median fitted leaf size (98) for a class created by insertion. Capacity splits start from the first principal direction of the new works (Boley 1998). Both sides need at least 5 works.
3. A split turns leaf C into the parent of two new classes: `C.01`, which keeps the core (C's earlier works and the new works near them) and is labelled "<C's label> (general)", and `C.02`, which takes the other group and is labelled from its works' most frequent title words.

4.2 Guarantee

Let S be a scheme and S' the scheme after inserting any sequence of works. Then:

- (G1) Existing classes are unchanged. Every class of S is in S' with the same code, label, vocabulary, note and parent, and the classes of S appear in S' in the same relative order.
- (G2) Only leaves are refined. Every class of S' that is not in S has a parent, that parent was a leaf class of S or is itself a class created by insertion, and the new class's code extends its parent's code (`C` to `C.01`, `C.02`).
- (G3) Existing works only move down. A work filed in class C of S is in C or in a descendant of C in S'; when C is split, its earlier works go to `C.01`.
- (G4) Shelf positions of existing works are kept. Because `C.01` opens C's run and `C.02` follows it, a work that was in C keeps its position relative to every work of S.

G1 and the parent clause of G2 are what `insert::check_stable` checks; `learned-scheme insert` runs it on every insertion and writes no output scheme if it fails. The test `holdout_insertion_keeps_every_existing_id_and_parent_lin` (`schemes/learned/tests/insertion.rs`) inserts the 500-book holdout and checks G1 class by class (code, parent, label, vocabulary, relative order), G2 including the code-extension clause, that exactly two classes appear per split, that every new work sits in a current leaf class, and the G3 clause that a split leaf's earlier works move only to a descendant. `insertion_is_deterministic` checks that the same inputs give byte-identical output, and `stability_check_rejects_changes_to_existing_classes` (`tests/insert_rules.rs`) checks that `check_stable` rejects a relabelled class, a new class under an existing internal class, and a reordering. G4 follows from the code grammar and the core-first construction; it is not separately tested beyond G3.

The consequence for a catalogue: S' is a conservative extension of S. Its codes and parent links are a superset of S's, an existing call number stays valid (it may become less specific than it could be, never wrong), and the inserted scheme takes the version `<version>.ins<count>`, so a client that compares versions sees only the split leaves as changed.

What the guarantee does *not* say: it bounds nothing about J after insertion, it does not keep the depth bound (a leaf can be split repeatedly), and it says nothing about label quality for the new classes. Section 8 returns to each of these.

5. Labels

No language model and no API was used. The 110 fitted classes were labelled by hand, by the AI coding assistant that built the prototype, from a label sheet (`learned-scheme build --sheet`). For every class the sheet shows its size, its top c-TF-IDF terms, and its most central titles (largest summed graph weight to the class's other works). c-TF-IDF is class-based TF-IDF as in BERTopic (Grootendorst 2022): tf(t, c) / tokens(c) * ln(1 + A / f(t)) over titles counted twice and abstracts, unigrams and adjacent bigrams, keeping terms present in a minimum share of the class's works (`src/terms.rs`). The top terms also become the class's vocabulary in the scheme file (source `ctfidf`), which is how shelfmark's lexical classifier reaches the class.

Each label line in `schemes/learned/labels.tsv` stores the class's top terms as a fingerprint, and the builder refuses a label whose fingerprint no longer matches, so a refit that moves classes cannot keep stale labels silently. `committed_scheme_is_valid_and_every_label_is_hand_written` checks that no class of the committed scheme carries an automatic label. Classes created by insertion get automatic labels (Section 4.1), and the 80% fit used for the held-out split uses automatic labels throughout.

We note the distinction from LLM taxonomy pipelines (Zeng et al. 2024; Wan et al. 2024): there the model proposes the structure; here the structure is fixed by the objective and labelling is a separate, auditable step. Whether hand labels written this way are good labels is a question for human evaluation that this draft does not answer (Section 8).

6. Results

All runs use the Paper 1 corpus (10,000 arXiv papers, `corpus.jsonl` sha256 `7685685d0de2...`), holdout (500 Open Library editions with LC call numbers, `holdout.jsonl` sha256 `3e057eb4a257...`), the published k = 20 graphs (corpus graph sha256 `759d3b8461a9...`) and the same harness. Paper 1 placements were regenerated with `profile:lexical@2`; all 19 placed and 5 gold scores equal the published ones (checked in `results.json` and again by `make_tables.py`). Lower is better for cost_norm, J and nav; higher is better for skill. Three kinds of placement appear: *gold* (the source's own labels, e.g. arXiv categories or LC call numbers), *shelfmark lexical@2* (each scheme reached through the same lexical classifier, using only labels and vocabulary), and *own* (the learned scheme's fitted assignment or its insertion procedure). Unplaced works hang under the root, so low coverage raises cost. J is not defined for the binary average-linkage tree, which is not a usable classification.

6.1 In sample (the corpus it was fitted to)

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
learned	own, in-sample (fitted on this corpus)	1.0000	0.1722	0.9760	0.2644	18.45	39.7	4.98	345
arxiv	gold (source labels)	1.0000	0.3333	0.7861	0.4250	18.33	36.5	3.90	797
learned	shelfmark lexical@2	0.9993	0.3794	0.7317	0.5347	31.08	33.5	4.38	1183
openalex-topics	shelfmark lexical@2	0.9732	0.6414	0.4228	0.7757	26.85	77.6	4.45	554
unesco-thesaurus	shelfmark lexical@2	0.9731	0.7210	0.3289	0.8544	26.68	146.7	4.90	535
ipc	shelfmark lexical@2	0.9799	0.7690	0.2724	0.9056	27.31	317.8	6.31	794
ilc2	shelfmark lexical@2	0.9999	0.8189	0.2135	0.9358	23.38	295.1	5.09	417
lcc-full	shelfmark lexical@2	0.9999	0.8228	0.2089	0.9372	22.86	657.2	6.06	121

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
acm-ccs	shelfmark lexical@2	0.7445	0.8306	0.1997	0.9493	23.73	84.9	3.61	877
fdc	shelfmark lexical@2	0.7696	0.8337	0.1961	0.9615	25.56	54.5	3.21	1103
eurovoc	shelfmark lexical@2	0.9737	0.8479	0.1794	0.9718	24.80	269.3	4.98	350
lcc	shelfmark lexical@2	0.8765	0.8487	0.1783	0.9755	25.36	129.0	4.08	661
arxiv	shelfmark lexical@2	0.4260	0.8912	0.1282	0.9929	20.34	14.5	2.21	1665
jel	shelfmark lexical@2	0.4786	0.9028	0.1146	0.9998	19.41	28.9	2.27	972
bk	shelfmark lexical@2	0.4806	0.9019	0.1157	1.0066	20.94	75.0	3.15	788
msc	shelfmark lexical@2	0.3916	0.9498	0.0592	1.0389	17.82	114.4	2.00	380
msc	gold (source labels)	0.1314	0.9844	0.0183	1.0605	15.20	461.3	1.39	25
jel	gold (source labels)	0.0002	1.0000	0.0000	1.0714	14.29	6.0	1.00	1
average linkage (binary reference, not a classification)	-	1.0000	0.1518	1.0000	-	-	-	-	2

Table 3. Corpus, n = 10,000. The first learned row is in sample by construction.

In sample the learned scheme reaches cost 0.1722, skill 0.9760: 97.6% of the average-linkage revenue with 110 named classes and 4 levels. arXiv's own author-chosen categories score 0.3333 (J 0.4250). The best published scheme reached through the lexical classifier is openalex-topics at 0.6414; the worst corpus-wide placed row is msc at 0.9498. This row is the objective the scheme was fitted to, evaluated on the data it was fitted on, so it is an upper bound on what the method shows, not evidence of generalization.

6.2 Shelfmark-placed

The second learned row is the scheme as reached by shelfmark's lexical classifier, like every published scheme in the placed rows: cost 0.3794, J 0.5347, coverage 99.93%. This removes the embedding from placement, but not the fit: the classes are cut along the corpus graph's clusters and their vocabulary is the corpus's own most distinctive terms. It is still in sample. The placed learned scheme also pays a higher navigation cost (31.08 bits against 18.45 for its own placements): the classifier concentrates works in fewer, larger classes (effective classes 33.5 against 39.7; largest class fan-out 1183 against 345).

6.3 Held out: fitted on 80%, tested on 20%

The split ranks ids by SHA-256 with seed `learned-split-2026-10-10`; the 2,000 test ids are committed in `bench/results/2026-10-10-learned/split/`. The 80% fit uses the same parameters with automatic labels and has 86 classes. The test works are then inserted as new works by nearest leaf centroid (0 splits occurred, so the scheme stayed at 86 classes). Everything in Table 4 is scored on the graph induced on the test works (average linkage 0.1024); every published scheme is restricted to the same works.

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
learned (fit on the other 80%)	own insertion, held out	1.0000	0.1722	0.9222	0.2528	16.10	33.9	5.00	69
learned (full fit)	own, in-sample	1.0000	0.1793	0.9144	0.2600	16.15	39.2	4.98	71
arxiv	gold (source labels)	1.0000	0.3424	0.7327	0.4220	15.92	35.7	3.90	167
learned (full fit)	shelfmark lexical@2	0.9995	0.3798	0.6910	0.4946	22.98	33.0	4.39	238
openalex-topics	shelfmark lexical@2	0.9750	0.6519	0.3878	0.7580	21.22	70.7	4.43	119
unesco-thesaurus	shelfmark lexical@2	0.9740	0.7192	0.3129	0.8236	20.89	134.4	4.89	127
ipc	shelfmark lexical@2	0.9815	0.7657	0.2611	0.8689	20.65	302.9	6.33	152
ilc2	shelfmark lexical@2	1.0000	0.8196	0.2010	0.9139	18.87	262.6	5.05	76
lcc-full	shelfmark lexical@2	0.9995	0.8325	0.1866	0.9264	18.77	566.6	6.09	30
acm-ccs	shelfmark lexical@2	0.7460	0.8346	0.1843	0.9283	18.74	84.1	3.60	164

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
fdc	shelfmark lexical@2	0.7730	0.8328	0.1863	0.9326	19.95	52.9	3.20	234
eurovoc	shelfmark lexical@2	0.9775	0.8411	0.1770	0.9359	18.94	250.6	5.00	75
lcc	shelfmark lexical@2	0.8755	0.8547	0.1619	0.9542	19.90	120.1	4.04	156
arxiv	shelfmark lexical@2	0.4185	0.8955	0.1164	0.9776	16.41	13.7	2.19	340
bk	shelfmark lexical@2	0.4825	0.8959	0.1159	0.9806	16.93	71.4	3.17	158
jel	shelfmark lexical@2	0.4780	0.9029	0.1082	0.9815	15.74	28.5	2.28	207
msc	shelfmark lexical@2	0.3870	0.9552	0.0499	1.0261	14.18	112.9	1.99	79
msc	gold (source labels)	0.1360	0.9805	0.0217	1.0431	12.51	269.7	1.41	11
jel	gold (source labels)	0.0000	1.0000	0.0000	1.0598	11.97	1.0	1.00	0

Table 4. Held-out test split, n = 2,000.

Held out, the learned scheme scores 0.1722 (J 0.2528). The full fit, which saw these works, scores 0.1793 on them, so there is no sign of overfitting at this scale. arXiv's categories score 0.3424; the best published scheme through the classifier, openalex-topics, 0.6519. Table 5 compares the fits on the whole graph.

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
learned (fit on 80%)	own, in-sample, 80% graph	1.0000	0.1665	0.9777	0.2570	18.11	33.7	5.00	259
learned (fit on 80%)	80% fitted + 20% inserted, whole corpus graph	1.0000	0.1720	0.9763	0.2641	18.42	33.8	5.00	328

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
learned (full fit)	own, in-sample, whole corpus graph	1.0000	0.1722	0.9760	0.2644	18.45	39.7	4.98	345

Table 5. The 80% fit in sample, the 80% fit with the 20% inserted, and the full fit, on the corpus graph.

With the test works inserted, the 80% fit scores 0.1720 (J 0.2641) on the whole corpus graph, against 0.1722 (J 0.2644) for the full fit. Insertion lost nothing measurable here. The caveat that matters is in Section 8.1: insertion files works by the same embedding the graph is built from.

6.4 Holdout: library books inserted as new works

The holdout is 500 Open Library editions with librarian-assigned LC call numbers, titles only, on its own k = 20 graph (average linkage 0.3014).

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
learned	own insertion (embedding nearest centroid)	1.0000	0.4432	0.7971	0.5416	19.69	15.1	8.79	93
lcc	gold (source labels)	0.9220	0.7076	0.4185	0.8006	18.59	35.0	4.20	89
lcc-full	gold (source labels)	0.9220	0.7057	0.4212	0.8043	19.71	85.2	5.19	68
lcc-full	shelfmark lexical@2	0.5660	0.9250	0.1074	0.9949	13.98	340.8	4.14	13
ilc2	shelfmark lexical@2	0.4560	0.9543	0.0655	1.0166	12.46	188.1	2.94	25
unesco-thesaurus	shelfmark lexical@2	0.1580	0.9888	0.0161	1.0429	10.82	93.8	1.65	8
learned	shelfmark lexical@2	0.0860	0.9915	0.0122	1.0443	10.57	26.1	1.34	5

scheme	placements	coverage	cost_norm	skill	J	nav log a=1	classes used	mean depth	max branching
lcc	shelfmark lexical@2	0.0840	0.9964	0.0052	1.0474	10.21	63.0	1.29	6
fdc	shelfmark lexical@2	0.0520	0.9981	0.0027	1.0486	10.10	31.1	1.14	4
bk	shelfmark lexical@2	0.0440	0.9985	0.0021	1.0492	10.14	30.8	1.23	7
average linkage (binary reference, not a classification)	-	1.0000	0.3014	1.0000	-	-	-	-	2

Table 6. Holdout, n = 500 books.

The books were inserted into the fitted scheme. The 110 classes became 126 through 8 leaf splits (3 for a distinct group, 5 for capacity), and every existing code, label and parent link was unchanged (Section 4.2). Table 7 lists the splits.

after insert no.	cause	split leaf	new class	its label (automatic)	new works in it	centroid cosine
45	distinct	01.02.01.04	01.02.01.04.02	harry / potter / adventures	12	0.205
70	distinct	01.02.01.04.01	01.02.01.04.02	arthur / cak' / chan'"	5	0.304
158	capacity	01.02.01.04.01	01.02.01.04.01	01.01.01.02 / man / power	53	0.780
244	capacity	01.02.01.04.02	01.02.01.04.02	01.01.01.02 / dog / power	42	0.788
355	distinct	01.01.01.04	01.01.01.04.02	amigos / como / dick	8	0.262
392	capacity	01.02.01.04.02	01.02.01.04.02	bell / 01.02.01.02 / beloved / blue	48	0.801
397	capacity	01.02.01.04.01	01.02.01.04.01	edition / 01.01.01.02 / holmes / sherlock	54	0.850
462	capacity	01.02.01.04.02	01.02.01.04.02	blood / 01.02.02.02 / body / cold	54	0.773

Table 7. Leaf splits while inserting the holdout (`insert-report.json`). Split leaves among the fitted classes: `01.01.01.04` (Language technology, society and bias), `01.02.01.04` (Image generation and editing).

Because the scheme was fitted on arXiv only, most books first land in one fitted leaf, `01.02.01.04` "Image generation and editing", the nearest centroid to short book titles; the first split, after 45 books, separates them from that class's papers, and later capacity splits divide them further. On the holdout graph the inserted scheme scores 0.4432 (skill 0.7971), below the librarians' call numbers (LCC gold 0.7076 at 92.2% coverage, full LCC schedules gold 0.7057). Through the lexical classifier the learned scheme places only 43 of the 500 books (8.6%) and scores 0.9915; the best placed scheme on the holdout is lcc-full at 0.9250 (56.6% coverage). We do not read the 0.4432 as a win over LCC; Section 8.1 explains why.

6.5 Sensitivity

36 fits of the full corpus with automatic labels cover depth bounds 3, 4, 6, lambda in 0.001, 0.002, 0.005, 0.01, 0.02, 0.05 and a in 1, 2 (`results.json`, `sweep`). Table 8 shows a = 1.

depth bound	lambda	J	cost_norm	nav (a=1)	classes	leaf classes	root children	median leaf
3	0.001	0.2049	0.1875	17.46	59	50	2	174
3	0.002	0.2224	0.1875	17.46	59	50	2	174
3	0.005	0.2747	0.1877	17.41	54	45	2	226
3	0.01	0.3619	0.1879	17.40	51	42	2	247
3	0.02	0.5361	0.1881	17.40	49	40	2	262
3	0.05	1.0513	0.2075	16.88	33	28	3	307
4	0.001	0.1902	0.1717	18.50	125	100	2	81
4	0.002	0.2087	0.1717	18.49	120	95	2	87
4	0.005	0.2644	0.1722	18.45	110	88	2	98
4	0.01	0.3565	0.1731	18.33	92	73	2	121
4	0.02	0.5359	0.1860	17.50	52	42	2	243
4	0.05	1.0513	0.2075	16.88	33	28	3	307
6	0.001	0.1783	0.1577	20.68	330	248	2	31
6	0.002	0.1990	0.1581	20.44	271	204	2	45
6	0.005	0.2594	0.1600	19.90	207	161	2	56
6	0.01	0.3550	0.1675	18.76	101	77	2	119
6	0.02	0.5359	0.1860	17.50	52	42	2	243
6	0.05	1.0513	0.2075	16.88	33	28	3	307

Table 8. Sensitivity of the fit to lambda and the depth bound, a = 1. The committed scheme is depth bound 4, lambda 0.005.

Raising lambda trades D for fewer levels and classes. The root has two children in 27 of the 36 fits; the 9 exceptions all have lambda of at least 0.02, and the widest root has 6 children.

6.6 Independent embeddings and a cost that sees false joins

Sections 6.1 to 6.4 score the scheme on the graph it was fitted to. Three later runs test whether its lead survives a different similarity, a corrected objective and a sweep of graph settings. All intervals in this section are paired item-bootstrap 95% percentile intervals over 1,000 draws.

A second and third embedding model. The corpus and holdout were re-embedded with two open models from other families, BAAI/bge-small-en-v1.5 and nomic-embed-text-v1.5, and graphs were built at k = 10, 20 and 50, giving 9 conditions with MiniLM (`bench/results/2026-10-10-embed`). The placements do not change; each is rescored on every graph. Table 9 counts the conditions in which the learned scheme leads the best other row of the same comparison (any Paper 1 scheme, gold or placed) and gives the tightest one.

comparison	learned first by skill	skill margin CI > 0	J margin CI > 0	tightest condition: skill margin [95% CI]
held out 20% (MiniLM fit, inserted)	9 of 9	9 of 9	9 of 9	bge-small-k10: 0.155 [0.122, 0.189]
in sample (full MiniLM fit)	9 of 9	9 of 9	9 of 9	bge-small-k50: 0.160 [0.151, 0.169]
500 holdout books, inserted	9 of 9	9 of 9	9 of 9	nomic-k10: 0.280 [0.214, 0.340]
placed by lexical@2	9 of 9	9 of 9	9 of 9	nomic-k10: 0.290 [0.277, 0.302]

Table 9. The learned scheme against the best other row in each of 9 graph conditions (three models, k = 10, 20, 50). Skill margin is learned skill minus the best other skill; J margin is the best other J minus learned J, so positive means the learned scheme is ahead on both. J here is the fitted objective, cost_norm + 0.005 N.

The edge survives the change of embedding: in every comparison and every condition the learned scheme ranks first by skill, and both margins have intervals above zero. The tightest case is the held-out split on bge-small-k10, a skill margin of 0.155 [0.122, 0.189]. The edge does shrink off the fit graph. On the held-out split at k = 20 the skill margin is 0.190 on MiniLM, 0.166 on bge-small and 0.167 on nomic, and held-out skill falls from 0.922 to 0.901 and 0.891. Refitting on each of the other two models gives the mirror image (Table 10): each fit scores best on its own model's graph, by 0.926 against 0.904 held-out skill on average. That difference is home advantage, and it is a small part of the margin.

learned fit	scored on minilm	scored on bge-small	scored on nomic
fitted on minilm	0.922	0.901	0.891
fitted on bge-small	0.913	0.938	0.909
fitted on nomic	0.899	0.912	0.918

Table 10. Held-out skill of each learned fit (rows) on each model's k = 20 papers graph (columns).

This test removes the circularity of the *scoring* graph only. The held-out and holdout works are still filed by their MiniLM vectors (Section 8.1).

A cost that sees false joins. Dasgupta cost charges for separating similar works and nothing for joining dissimilar ones. A blind LLM judge of librarian-style pair decisions agrees with the cost's flag at Cohen's $\kappa = 0.27$ on 953 pairs (Cohen 1960); adding a false-join term raises that to 0.48 (Paper 1, Sections 3.7 and 5.6 to 5.7). For a seeded sample of non-edge pairs whose cosine is below the graph's mean pair cosine τ, weighted by $d = \tau - \cos$, the specificity form is $\text{FJs} = \sum d \ln(n/N(\text{lca}))/(\ln n \sum d)$ and the outside-mass form is $\text{FJ} = \sum d\,(n - N(\text{lca}))/(n \sum d)$, one minus the normalised dissimilarity objective of Cohen-Addad et al. (Cohen-Addad et al. 2019). The corrected objective is $J' = C + \text{FJs} + 0.002\,N$. Its weight on N is Paper 1's, not the 0.005 the scheme was fitted with. For the learned scheme placed by the lexical classifier, the judge's agreement does not change with the term (κ 0.33 to 0.30, change -0.03 [-0.20, +0.14]): the judge rarely finds a false join in it.

Table 11 compares the learned scheme, with its own placements, against the best non-learned run under each form, on the 15 graphs of the false-join rescoring (`bench/results/2026-10-11-falsejoin`). "Best other" is chosen by point value among all non-learned runs of that universe, including gold Dewey and LCC numbers on the books, before the interval is computed.

universe	model	k	best other by J'	J' learned - best [95% CI]	best other by C + FJ	C + FJ learned - best [95% CI]
3,000 books	minilm	10	lcc-gold	-0.016 [-0.043, +0.007]	lcc-gold	+0.140 [+0.111, +0.167]
3,000 books	minilm	20	ddc-section	-0.105 [-0.129, -0.083]	lcc-full-gold	+0.045 [+0.020, +0.068]
3,000 books	minilm	50	ddc-d2	-0.152 [-0.171, -0.134]	lcc-gold	-0.037 [-0.059, -0.016]
3,000 books	bge-small	10	lcc-gold	+0.018 [-0.008, +0.040]	lcc-gold	+0.183 [+0.155, +0.209]
3,000 books	bge-small	20	ddc-section	-0.052 [-0.075, -0.032]	lcc-full-gold	+0.092 [+0.068, +0.115]
3,000 books	bge-small	50	ddc-d2	-0.097 [-0.116, -0.079]	ddc-gold	+0.015 [-0.007, +0.038]
3,000 books	nomic	10	lcc-gold	+0.036 [+0.010, +0.060]	lcc-gold	+0.195 [+0.168, +0.223]
3,000 books	nomic	20	ddc-section	-0.058 [-0.082, -0.036]	lcc-full-gold	+0.098 [+0.072, +0.123]
3,000 books	nomic	50	ddc-section	-0.108 [-0.129, -0.090]	lcc-gold	+0.010 [-0.012, +0.033]
10,000 papers (in sample)	minilm	20	fdc-arxiv-nested	-0.136 [-0.145, -0.128]	arxiv-gold	-0.041 [-0.051, -0.031]
10,000 papers (in sample)	bge-small	20	fdc-arxiv-nested	-0.115 [-0.122, -0.106]	arxiv-gold	-0.046 [-0.056, -0.036]
10,000 papers (in sample)	nomic	20	fdc-arxiv-nested	-0.116 [-0.124, -0.108]	arxiv-gold	-0.036 [-0.047, -0.027]
500 holdout books (title only)	minilm	20	lcc-pipeline	-0.264 [-0.309, -0.221]	lcc-full-gold	-0.249 [-0.306, -0.192]
500 holdout books (title only)	bge-small	20	fdc-pipeline	-0.181 [-0.226, -0.137]	fdc-work	-0.110 [-0.168, -0.053]
500 holdout books (title only)	nomic	20	fdc-work	-0.184 [-0.235, -0.126]	fdc-work	-0.134 [-0.198, -0.066]

Table 11. Learned scheme minus the best other run under J′ and under C + FJ (both at $\beta = 1$). Negative means the learned scheme is ahead. Books and holdout are inserted as new works; the papers are the fitted sample.

On the papers and the title-only holdout, the learned scheme leads under both forms in every graph (3 of 3 and 3 of 3 under J′). On the 3,000 books it does not hold. Under J′ it leads in 6 of 9 graphs, ties in 2 and is behind in 1; the ties and the loss are against LCC's librarian numbers on the k = 10 graphs. Under the outside-mass form it is behind in 6 of 9, ties in 2 and leads in 1. The outside-mass form charges a shallow join almost as much as a deep one, so it rewards a wide top level (`bench/results/2026-10-11-falsejoin/README.md`). The learned scheme has a binary root (Section 8.3), so every dissimilar pair that lands in the same half of it pays at least half the maximum.

The 27-setting book sweep. The book comparison was repeated in 27 settings: three models, three values of k and

three graph thresholds (`bench/results/2026-10-11-book-stability`). The claim that the learned scheme beats Dewey sections under J' has the claimed sign in 19 of 27 settings and is significantly reversed in 6. By that run's own rule the claim is not stable.

Verdict. The learned scheme's edge survives an independent embedding. On arXiv papers it also survives the false-join correction. On library books, which it was not fitted to, it does not survive: under the corrected objective its lead over librarian-assigned Dewey and LCC numbers depends on the graph setting, and under the outside-mass form it is usually behind.

7. Discussion

What the prototype shows. Under a stated objective, a fitted order-preserving scheme with 110 hand-labelled classes scores well below every published scheme on that objective with its own placements, in sample and on a held-out split. Through the same lexical classifier as the published schemes it is still below every published scheme's placed row (0.3794 against 0.6414 for the best), but above arXiv's own author-chosen categories (0.3333). Most of that gap is expected: the published schemes were not built for this corpus or this similarity, and several are domain schemes on foreign ground. The useful result is not the ranking but the reference point. A hand-built scheme can now be described as "this far from a fitted scheme with the same number of levels on the same graph", and the cost of each constraint (order preservation, the depth bound, stable insertion) can be measured rather than argued.

What survives the checks. The lead holds when the scoring graph comes from another embedding model, and part of it, a few hundredths of skill, is home advantage (Section 6.6). It does not hold on books once false joins are charged. A fitted scheme that is excellent on its own kind of material and only level with librarian numbers on books is a reference point, not a replacement.

The price of order preservation is not yet measured. The DP collapse and the local search work only within one fixed shelf order, and the fitted tree keeps 97.6% of the average-linkage revenue. That loss mixes two effects: collapsing a binary tree to a bounded-shape class tree, and doing so without reordering works. Any binary tree has a compatible leaf order, so the order constraint costs nothing before the collapse; whether it costs much after it needs a bounded-shape fit without the order constraint, which we have not run. Whether the answer holds for a heterogeneous collection is open.

Stability as a design constraint. Separating batch fitting from incremental growth gives a guarantee a cataloguer can rely on (Section 4.2) at the price of drift: the inserted scheme is no longer J-optimal and its depth grows where new material concentrates. A periodic refit would restore optimality but break identifiers. A principled middle ground, an objective that charges for every renumbered work, is the open problem we would most like to see addressed (research report 2 found no prior formulation).

Towards a general classification. A general scheme needs a general corpus. The corpus here is papers from arXiv's top-level archives; a library's collection is mostly books, much of it in the humanities, arts and social sciences, with titles and subject headings rather than abstracts. The method transfers (it needs only a similarity graph and text for labels); the scheme does not, as the holdout shows.

8. Limitations

8.1 Embedding circularity flatters the held-out and holdout numbers

The graph is built from sentence-transformers/all-MiniLM-L6-v2 embeddings, and insertion files a new work at the nearest leaf centroid in the *same* embedding space. A new work is therefore placed by exactly the geometry it is later scored on. This is the circularity the benchmark design warns about (DESIGN.md, finding 6) and it favours the learned rows in Tables 4 and 6. It is worst on the holdout, where both the graph and the insertion see only title embeddings. The published schemes' placed rows, and the learned scheme's own placed rows, use the lexical classifier and do not share this advantage. Section 6.6 removes the circularity of the scoring graph by rescoring on graphs from two other embedding models, and the lead survives. It does not remove the circularity of filing: inserted works are still placed by their MiniLM vectors, and a similarity that is not an embedding at all

(citation coupling or subject-heading overlap; DESIGN.md section 2.2) has been used only to validate the metric, not to score this scheme.

8.2 Fitted on arXiv only

The scheme has seen only arXiv papers. Its top level splits computing from mathematics and the physical sciences; it has no class for the humanities, law, arts or literature until insertion creates one, and then a weak one. It is a general *method*, not yet a general classification. The arXiv sample itself is proportional to archive size with a floor per archive (`bench/corpus/README.md`), and computer science is its largest part: 39.7% of the papers have a `cs` primary category.

8.3 A binary root under this objective

The root has 2 children in the committed scheme and in 27 of 36 sweep fits. Dasgupta cost prefers a binary split near the top, where most cross-cluster weight sits, and the navigation term charges only per level, so it cannot prefer a wide first menu. Library schemes have wide top levels (LCC's first level is its set of lettered main classes (Library of Congress 2026b)) for reasons of browsing and memorability that J does not model. A top-level fan-out prior, or a navigation cost with a linear scan term (the harness's `nav_scan`), may change this; we have not tested either.

8.4 Weak automatic labels for inserted classes

Classes created by insertion are labelled from their works' most frequent title words. Table 7 shows the result: labels such as "Harry / potter / adventures" for the first class of books, and a chain of "(general)" core classes that repeat their parent's label down 9 levels. These are not usable as catalogue captions. The hand labels for fitted classes were written by an AI coding assistant from the label sheet, not by a cataloguer, and have not been evaluated by people, nor scored with any of the taxonomy-quality attributes in the literature (Unterkalmsteiner and Abdeen 2023).

8.5 Classifier coverage on book titles

The class vocabulary is mined from arXiv titles and abstracts, so shelfmark's lexical classifier rarely matches book titles: 43 of 500 holdout books (8.6%) were placed. The non-circular comparison on books (Section 8.1) is therefore uninformative for this scheme until its vocabulary covers books.

8.6 Graph-hash non-reproducibility with near-tied edges

Rebuilding the published graphs from the pinned embedding weights (`model.onnx` sha256 `bbd7b466f6d5...`) on a different ONNX Runtime version gives edge weights that differ by floating-point noise and, in a few places, a different choice between near-tied nearest neighbours; the rebuilt graph's hash therefore differs from the published one (`bench/results/2026-10-10-learned/REPRODUCE.md`). The fit uses only the published graph, so the scheme rebuilds byte for byte, but anyone regenerating the graph from text will get a slightly different graph and possibly a slightly different scheme. The vectors enter insertion only through i8-quantized centroids, whose quantization step is coarser than the noise. The graph file, not the embedding pipeline, is the reproducible artefact.

8.7 Other limits

- Insertion does not keep the depth bound: holdout insertion produced classes 9 levels deep and a mean work depth of 8.79 in the harness's tree (Table 6).
- One corpus to fit. Tables 1 to 8 are point values on the fit graph; the paired intervals are in Section 6.6. A permutation null for the learned rows (DESIGN.md section 2.5) has not been run.
- The false-join validity check rests on one LLM judge, and it does not raise agreement with citation or subject-heading signals (Paper 1, Section 5.7).

- No user study: N is a model of navigation cost (Hick 1952; Hyman 1953), not a measurement. User studies of menu depth and breadth disagree on the best shape: speed and accuracy improved with breadth in one (Snowberry, Parkinson, and Sisson 1983), others found an optimum at moderate breadth (Miller 1981; Larson and Czerwinski 1998), and none of these was run on a library classification, so they do not calibrate N.
- The comparison mixes gold rows (a source's own labels), placed rows (a classifier's output) and own rows (fitted or inserted); they answer different questions and should not be read as one ranking.

9. Release as CC0 open data

`schemes/learned/learned.json` (`scheme:learned@2026-10-10`, sha256 `379f746e99ad...`), its hand-written labels (`labels.tsv`), and the leaf centroids and quantized holdout vectors in `schemes/learned/data/` are dedicated to the public domain under CC0 1.0 (Creative Commons, n.d.). The builder code is MIT OR Apache-2.0 like the rest of the repository. CC0 is chosen so that the scheme, unlike DDC, can be copied into catalogues, crosswalks and derived schemes without conditions, and so that it combines with other CC0 sources: arXiv metadata (arXiv, n.d.) and Open Library's catalogue data, which Open Library describes as dedicated to the public domain and asks contributors to share under CC0 1.0 (Open Library, n.d.). The scheme validates against the repository's scheme schema (`shelfmark scheme validate --path schemes/learned/learned.json`), and `sources.lock.json` records the sha256 of every input. The scheme identifiers are versioned (`learned@2026-10-10`, `learned@2026-10-10.ins500` for the holdout insertion), and the stability guarantee of Section 4 is part of the release: a future version that grows by insertion keeps every code of this one.

Repository: https://github.com/davidawad/shelfmark (the scheme is in `schemes/learned/`).

10. Reproducibility

- Inputs. Corpus, holdout, the published $k = 20$ graphs and the scheme are committed with their sha256 values; the graphs were built with sentence-transformers/all-MiniLM-L6-v2 weights revision `d13954661f83`.
- Fit. `cargo run --release --manifest-path schemes/learned/Cargo.toml -- build ...` (commands in `schemes/learned/README.md`) rebuilds `learned.json` byte for byte from committed files in seconds; the test `rebuild_reproduces_the_committed_scheme_and_matches_the_harness` enforces this, and `sources_lock_and_registry_match_the_committed_files` checks the input hashes.
- Benchmark. `bench/results/2026-10-10-learned/REPRODUCE.md` gives every command, including the Paper 1 placement rerun, embedding the vectors for the centroids, and `crates/bench/scripts/learned_benchmark.py`. The results record the tool commit (`6b9ff9de9a66`) and binary hashes.
- Determinism. The fit is seeded (20261010) and single-threaded; insertion is deterministic (tested). The one non-reproducible step is regenerating the graph from text (Section 8.6).
- This paper. `python3 paper/paper2/scripts/make_tables.py` regenerates `paper.md` and `tables/` from the JSON; `--check` fails if they are stale. The script cross-checks J against its two terms for every row, the placed rows against the Paper 1 rerun, and the fit report against the harness.

References

See `references.bib`. Citations appear in the text as pandoc citation keys.

arXiv. n.d. "arXiv License Information." https://info.arxiv.org/help/license/index.html.

BARTOC. n.d. "Basisklassifikation (Basic Classification, BK)." BARTOC record. https://bartoc.org/en/node/18785.

Boley, Daniel. 1998. "Principal Direction Divisive Partitioning." *Data Mining and Knowledge Discovery* 2 (4): 325–44. https://doi.org/10.1023/A:1009740529316.

Carlsson, Gunnar, and Facundo Mémoli. 2010. "Characterization, Stability and Convergence of Hierarchical Clustering Methods." *Journal of Machine Learning Research* 11 (47): 1425–70. https://www.jmlr.org/papers/v11/carlsson10a.html.

Chami, Ines, Albert Gu, Vaggos Chatziafratis, and Christopher Ré. 2020. "From Trees to Continuous Embeddings and Back: Hyperbolic Hierarchical Clustering." In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 15065–76. https://papers.nips.cc/paper_files/paper/2020/hash/ac10ec1ace51b2d973cd87973a98d3ab-Abstract.html.

Charikar, Moses, and Vaggos Chatziafratis. 2017. "Approximate Hierarchical Clustering via Sparsest Cut and Spreading Metrics." In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 841–54. https://doi.org/10.1137/1.9781611974782.53.

Cohen, Jacob. 1960. "A Coefficient of Agreement for Nominal Scales." *Educational and Psychological Measurement* 20 (1): 37–46. https://doi.org/10.1177/001316446002000104.

Cohen-Addad, Vincent, Varun Kanade, Frederik Mallmann-Trenn, and Claire Mathieu. 2019. "Hierarchical Clustering: Objective Functions and Algorithms." *Journal of the ACM* 66 (4): 1–42. https://doi.org/10.1145/3321386.

Comaromi, John P. 1976. "Conception and Development of the Dewey Decimal Classification." *Knowledge Organization* 3 (1): 11–15. https://doi.org/10.5771/0943-7444-1976-1-11.

Creative Commons. n.d. "CC0 1.0 Universal Public Domain Dedication." https://creativecommons.org/publicdomain/zero/1.0/.

Dasgupta, Sanjoy. 2016. "A Cost Function for Similarity-Based Hierarchical Clustering." In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, 118–27. https://doi.org/10.1145/2897518.2897527.

Fraser, Craig. 2020. "Mathematics in Library and Review Classification Systems: An Historical Overview." *Knowledge Organization* 47 (4): 334–56. https://doi.org/10.5771/0943-7444-2020-4-334.

Grootendorst, Maarten. 2022. "BERTopic: Neural Topic Modeling with a Class-Based TF-IDF Procedure." https://arxiv.org/abs/2203.05794.

Hick, W. E. 1952. "On the Rate of Gain of Information." *Quarterly Journal of Experimental Psychology* 4 (1): 11–26. https://doi.org/10.1080/17470215208416600.

Hjørland, Birger. 2025. "Dewey Decimal Classification (DDC)." *Em Questão* 31: e–145068. https://doi.org/10.1590/1808-5245.31.145068.

Hu, T. C., and A. C. Tucker. 1971. "Optimal Computer Search Trees and Variable-Length Alphabetical Codes." *SIAM Journal on Applied Mathematics* 21 (4): 514–32. https://doi.org/10.1137/0121057.

Huffman, David A. 1952. "A Method for the Construction of Minimum-Redundancy Codes." *Proceedings of the IRE* 40 (9): 1098–1101. https://doi.org/10.1109/JRPROC.1952.273898.

Hyman, Ray. 1953. "Stimulus Information as a Determinant of Reaction Time." *Journal of Experimental Psychology* 45 (3): 188–96. https://doi.org/10.1037/h0056940.

ISKO Italia. n.d. "Integrative Levels Classification, Edition 2." https://www.iskoi.org/ilc/.

Kleinberg, Jon M. 2002. "An Impossibility Theorem for Clustering." In *Advances in Neural Information Processing Systems 15 (NIPS 2002)*. https://papers.nips.cc/paper_files/paper/2002/hash/43e4e6a6f341e00671e123714de019a8-Abstract.html.

Knuth, Donald E. 1971. "Optimum Binary Search Trees." *Acta Informatica* 1 (1): 14–25. https://doi.org/10.1007/BF00264289.

Larson, Kevin, and Mary Czerwinski. 1998. "Web Page Design." In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems - CHI '98*, 25–32. ACM Press. https://doi.org/10.1145/274644.274649.

Library of Congress. 2026a. "About: LC Linked Data Service." https://id.loc.gov/about/.

———. 2026b. "Library of Congress Classification PDF Files." https://www.loc.gov/aba/publications/FreeLCC/freelcc.html.

Miller, Dwight P. 1981. "The Depth/Breadth Tradeoff in Hierarchical Computer Menus." *Proceedings of the Human Factors Society Annual Meeting* 25 (1): 296–300. https://doi.org/10.1177/107118138102500179.

Moseley, Benjamin, and Joshua R. Wang. 2023. "Approximation Bounds for Hierarchical Clustering: Average Linkage, Bisecting k-Means, and Local Search." *Journal of Machine Learning Research* 24 (1): 1–36. https://jmlr.org/papers/v24/18-080.html.

Ockerbloom, John Mark. n.d. "Free Decimal Correspondence." Everybody's Libraries. https://everybodyslibraries.com/free-decimal-correspondence/.

OCLC. n.d. "Dewey Services – Resources." https://www.oclc.org/en/dewey/resources.html.

Open Library. n.d. "Using Open Library Data." Open Library Help Center FAQ. https://openlibrary.org/help/faq/using.

OpenAlex. 2026. "Topics." OpenAlex Help Center. https://help.openalex.org/hc/en-us/articles/24736129405719-

Topics.
Priem, Jason, Heather Piwowar, and Richard Orr. 2022. "OpenAlex: A Fully-Open Index of Scholarly Works, Authors, Venues, Institutions, and Concepts." https://arxiv.org/abs/2205.01833.
Publications Office of the European Union. n.d. "EuroVoc." EU Vocabularies. https://op.europa.eu/en/web/eu-vocabularies/th-dataset/-/resource/dataset/eurovoc.
Reimers, Nils, and Iryna Gurevych. 2019. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3980–90. https://doi.org/10.18653/v1/D19-1410.
sentence-transformers. n.d. "All-MiniLM-L6-V2." Hugging Face model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.
Shen, Jiaming, Zhihong Shen, Chenyan Xiong, Chi Wang, Kuansan Wang, and Jiawei Han. 2020. "TaxoExpan: Self-Supervised Taxonomy Expansion with Position-Enhanced Graph Neural Network." In *Proceedings of the Web Conference 2020*, 486–97. https://doi.org/10.1145/3366423.3380132.
Snowberry, Kathleen, Stanley R. Parkinson, and Norwood Sisson. 1983. "Computer Display Menus." *Ergonomics* 26 (7): 699–712. https://doi.org/10.1080/00140138308963390.
Traag, Vincent A., Ludo Waltman, and Nees Jan van Eck. 2019. "From Louvain to Leiden: Guaranteeing Well-Connected Communities." *Scientific Reports* 9: 5233. https://doi.org/10.1038/s41598-019-41695-z.
UNESCO. n.d. "UNESCO Thesaurus: About." https://vocabularies.unesco.org/en/about.
Unterkalmsteiner, Michael, and Waleed Abdeen. 2023. "A Compendium and Evaluation of Taxonomy Quality Attributes." *Expert Systems* 40 (1). https://doi.org/10.1111/exsy.13098.
Waltman, Ludo, and Nees Jan van Eck. 2012. "A New Methodology for Constructing a Publication-Level Classification System of Science." *Journal of the American Society for Information Science and Technology* 63 (12): 2378–92. https://doi.org/10.1002/asi.22748.
Wan, Mengting, Tara Safavi, Sujay Kumar Jauhar, Yujin Kim, Scott Counts, Jennifer Neville, Siddharth Suri, et al. 2024. "TnT-LLM: Text Mining at Scale with Large Language Models." In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 5836–47. https://doi.org/10.1145/3637528.3671647.
Wang, Wenhui, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers." https://arxiv.org/abs/2002.10957.
Zeng, Qingkai, Yuyang Bai, Zhaoxuan Tan, Shangbin Feng, Zhenwen Liang, Zhihan Zhang, and Meng Jiang. 2024. "Chain-of-Layer: Iteratively Prompting Large Language Models for Taxonomy Induction from Limited Examples." In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 3093–3102. https://doi.org/10.1145/3627673.3679608.
Zhang, Chao, Fangbo Tao, Xiusi Chen, Jiaming Shen, Meng Jiang, Brian Sadler, Michelle Vanni, and Jiawei Han. 2018. "TaxoGen: Unsupervised Topic Taxonomy Construction by Adaptive Term Embedding and Clustering." In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2701–9. https://doi.org/10.1145/3219819.3220064.